UFOP — BCC362 — Sistemas Distribuídos

TP01: primeiro contato com Hazelcast

Aplicação: soma distribuída dos números de 1 até N.

Integrantes: preencher nomes e matrículas (até três).

Versões: Java 17 e Hazelcast 5.6.0, fixada para reproduzir o trabalho.

1. Proposta e relação com o enunciado

A proposta é implementar um exemplo básico já encontrado no site do framework e utilizá-lo para entender distribuição de dados e execução em cluster. Hazelcast está na lista de frameworks do enunciado. Aplicam-se os itens de descrição, arquitetura, aplicação e perguntas; a exigência específica de instalação de middleware com VMs/orquestradores não se aplica como obrigação dessa modalidade.

O ponto de partida é o exemplo SumTask da documentação oficial de Java Executor Service [1]. Cada membro soma os valores das chaves que lhe pertencem, e um cliente soma as respostas. O algoritmo foi mantido simples. Configuração, inspeção, benchmark e acompanhamento são recursos de apoio à demonstração, separados da classe SomaTask.

2. Para que serve o Hazelcast?

Hazelcast oferece armazenamento e processamento distribuídos em memória. Neste trabalho usamos IMap, um mapa chave–valor distribuído, e IExecutorService, que envia tarefas Java para os membros. Isso permite trabalhar com dados compartilhados e executar cálculo em mais de um processo. [1,2]

Aplicações possíveis incluem cache, sessões, contadores e estatísticas operacionais. Casos comerciais relatados pelo fornecedor incluem British Gas/Hive em IoT e BNP Paribas Bank Polska em serviços bancários. O caso Hive cita um cluster de 20 nós; o caso bancário relata 84 mil requisições por segundo em produção. São relatos do fornecedor, não medições do nosso trabalho nem garantia do desempenho atual desses sistemas. [4,5]

Não é uma solução que elimina os custos de rede, memória ou coordenação. Backup em RAM não é persistência em disco. Também não é necessário usar um cluster para calcular uma soma pequena: este problema foi escolhido para aprendizado, e não para alegar que a distribuição é a melhor forma de somar uma sequência conhecida.

Maior aplicação e quantidade de máquinas

Não foi localizada evidência suficiente para apontar a maior implantação mundial ou seu tamanho atual. O caso comercial Hive fornece um exemplo de 20 nós. Separadamente, um benchmark histórico do fornecedor usando Jet 4.3 relata 45 nós EC2, 720 vCPUs, 1 bilhão de eventos por segundo e latência p99 de 26 ms. Benchmark não é aplicação comercial nem prova de recorde mundial. Esses números não descrevem a capacidade do exemplo de soma. Nós também não devem ser confundidos automaticamente com máquinas físicas. [4,6]

3. Como funciona internamente?

O cliente se conecta ao cluster, escreve números no mapa e pede o cálculo. Os membros armazenam partições e executam tarefas. Cada chave é associada a uma partição por hash; uma tabela indica o membro proprietário. A configuração mantém as 271 partições padrão e uma cópia síncrona dos dados em outro membro quando possível. [2,3]

Há participação distribuída dos membros, mas existem funções de coordenação, como as relacionadas à tabela de partições. Não se deve afirmar que não existe nenhuma coordenação interna. A aplicação também tem um cliente central para reunir as respostas; sua queda interrompe o cálculo em andamento. [3]

Camada	Responsabilidade neste trabalho
Aplicação	Carregar 1..N, pedir a soma, juntar respostas e conferir
Serviços Hazelcast	IMap armazena dados; IExecutorService executa SomaTask
Gestão do cluster	Descoberta TCP, membros, detecção de saída e migração de partições
Infraestrutura	JVM, threads, RAM, serialização e comunicação TCP

Uma cópia em outro membro permite recuperar os dados quando um membro cai, desde que a cópia exista e esteja sincronizada. Com um único membro não há redundância em outro membro. Ao entrar ou sair um membro, os dados são redistribuídos. Os experimentos esperam as cópias ficarem seguras antes da falha. [2]

4. Algoritmo escolhido e origem

A soma é uma redução: vários valores são combinados em um resultado. É útil como base para totais e estatísticas. A operação é associativa para os valores inteiros utilizados, então podemos somar partes e depois juntar as parcelas. O exemplo implementado adapta SumTask da seção “Implementing a Callable Task” da documentação oficial. Não usamos a API histórica MapReduce nem um pipeline Jet. [1]

A sequência é simples: o cliente grava números de 1 até N; cada membro percorre suas chaves primárias locais; uma tarefa retorna a soma local; o cliente recebe uma parcela de cada membro e calcula o total. A fórmula N(N+1)/2 serve somente para validar o cálculo distribuído.

As adaptações são nomes em português, chaves Integer e valores Long, combinação das parciais no cliente, entrada determinística e comandos de demonstração. Long é necessário porque a soma até 100000 excede o limite de Integer.

Exemplo didático

Para N=10, os valores são 1 a 10 e a soma é 55. Se três membros tivessem parcelas 12, 15 e 28, o cliente calcularia 12+15+28=55. Essa divisão é apenas ilustrativa: as parcelas reais dependem do hash e da distribuição do cluster.

Desafios e limites

O trabalho aritmético total é O(N), e a combinação de p parciais é O(p). Entretanto, cada consulta ao mapa tem custos adicionais, mesmo para uma chave local; serialização, invocação, agendamento e coordenação impedem tratar O(N/p) como previsão exata de tempo. A carga guarda N pares e uma cópia adicional quando há membros suficientes. O cliente materializa a entrada e o conjunto de chaves de cada membro também usa memória.

O conjunto deve permanecer imutável durante a soma. localKeySet não é uma fotografia transacional durante migração: uma rodada pode falhar ou contar de forma inconsistente se a topologia mudar. A aplicação verifica a composição do cluster antes/depois e a resposta conhecida; rejeita uma rodada divergente. Isso não oferece garantia universal de integridade para quaisquer dados: conjuntos distintos podem ter a mesma soma.

5. Falha e expansão de maneira simples

O mapa tem um backup síncrono. Isso protege os dados contra a perda de um membro em um cluster previamente seguro, não todos os tipos de falha. O executor convencional pode perder uma tarefa em andamento. O exemplo não implementa recuperação por partes ou persistência de tarefas.

O modo acompanhar mantém o mesmo cliente aberto e realiza novas somas. Se uma rodada falhar, ele informa o problema; depois da reorganização, uma rodada seguinte refaz a soma inteira. A classe SomaTask apenas lê dados, então repetir a operação não incrementa algum total compartilhado. Não afirmamos execução exatamente uma vez nem continuidade sem pausa.

Na expansão, um novo membro se conecta pelas seeds, recebe partições e passa a participar das rodadas seguintes. O cliente de acompanhamento não precisa ser reiniciado. Tarefas em andamento não são

magicamente transferidas para o novo membro.

Não há persistência, TLS, autenticação ou proteção de split-brain configurados no laboratório. Perda completa do host, falha de dois membros antes da recomposição e queda do cliente não são cobertas. JVMs locais compartilham o domínio de falha físico; para proteção contra queda de máquina é necessário separar membros e cópias em máquinas apropriadas.

6. Experimentos e validação

A execução usa JVMs separadas em um host Linux/WSL, Java 17, heap máximo de 768 MiB por processo e ActiveProcessorCount=2. Isso demonstra comunicação distribuída por TCP, mas não é uma medição de três máquinas independentes.

A suíte testa N=0, N=1 e N=10, rejeição de N negativo e de N incompatível com os dados. Depois mede 1000, 10000 e 100000 números com 1, 2 e 3 membros. Para cada combinação, uma soma aquece o ambiente e três somas são registradas: 27 medições. A carga é medida uma vez por combinação e também reportada separadamente.

Finalmente, um único cliente de acompanhamento fica ativo enquanto o processo do membro 5703 é encerrado abruptamente e o membro 5704 é iniciado. Os logs devem comprovar a volta a respostas válidas com dois e depois três membros, com o mesmo PID de cliente. Os detalhes numéricos e o resultado efetivo constam em RESULTADOS.md, incorporado à versão PDF.

O tempo de soma inclui submissão, cálculo e espera das respostas, além da linha de início impressa; não inclui a conferência final nem a impressão das parciais. O tempo de carga inclui limpeza do mapa anterior, criação do mapa Java, envio e a mensagem de carga. Inicialização e conexão ficam fora. Não é um benchmark de produção: poucas repetições, ordem fixa, hardware compartilhado e tempo de aquecimento podem influenciar os resultados.

Aceleração $S(p)=T(1)/T(p)$ compara a mesma entrada com um e p membros. Eficiência $E(p)=S(p)/p$. Não foi criado um benchmark sequencial separado: o caso de um membro continua sendo Hazelcast, e não deve ser chamado de execução Java sequencial pura. Aumento de N com p fixo mostra crescimento do input; aumentar p com N fixo avalia escalabilidade forte.

7. Organização e apresentação

SomaTask.java contém a tarefa; Cliente.java contém carga e redução, seguidas de comandos de apoio; Main.java configura os membros; EstadoTask.java consulta o estado do cluster. O guia explica a execução manual, a automação e o que fazer se precisar instalar Java/WSL. A execução em máquinas remotas é uma alternativa descrita no guia, não declarada como executada.

Os slides foram refeitos em LaTeX/Beamer. Há fonte editável, PDF e notas de fala. A apresentação reserva cerca de cinco minutos para tecnologia, cinco para arquitetura e cinco para a aplicação/demonstração. Em seguida há dez minutos para perguntas. O questionário concreto não foi fornecido no PDF; as perguntas de preparação são apenas estudo.

Referências

A numeração [1–8] é detalhada em REFERENCIAS.md, incluída ao final do PDF. O código do exemplo é pequeno o suficiente para ser apresentado e explicado, sem exigir estudar a automação de testes como parte do algoritmo.

Resultados da soma distribuída

Campanha: 20260918-182626. Resultado: ALL_TESTS_PASSED.

Desempenho

JVMs separadas em um host Linux/WSL, Java 17.0.19, Hazelcast 5.6.0. Heap máximo 768 MiB por processo e ActiveProcessorCount=2. Uma tarefa por membro. Uma rodada de aquecimento descartada e três medições por combinação (27 medições). Não há atraso artificial. Mediana e intervalo mínimo–máximo abaixo.

N	Membros	Carga ms	Soma ms	Min–máx ms	Carga+soma ms	S(p)	E(p)
1000	1	112.69	104.21	88.17–105.53	216.90	1.00	1.00
1000	2	303.78	68.35	65.62–69.50	372.13	1.52	0.76
1000	3	214.17	40.47	39.80–43.01	254.64	2.58	0.86
10000	1	101.81	692.00	682.75–749.69	793.81	1.00	1.00
10000	2	160.34	347.20	333.50–389.60	507.54	1.99	1.00
10000	3	133.45	290.33	274.53–327.71	423.78	2.38	0.79
100000	1	227.63	6432.57	6188.06–7057.56	6660.20	1.00	1.00
100000	2	230.23	2921.74	2700.13–3237.70	3151.97	2.20	1.10
100000	3	207.84	2537.03	2455.89–3014.42	2744.88	2.54	0.85

S(p)=T(1)/T(p); E(p)=S(p)/p. A referência de um membro também usa Hazelcast, não é uma execução Java sequencial pura. Carga é medida uma vez por combinação e reutilizada na soma carga+processamento. O tempo de carga inclui limpeza, geração e envio; tempo de soma inclui submissão, execução e recebimento. Inicialização/conexão e validação final ficam fora.

Interpretação

Os resultados são observações deste host e desta campanha curta. Mais membros não garantem proporcionalidade: cada número ainda passa pela abstração IMap; threads, rede local, serialização e memória têm custo. JIT, coleta de lixo e ordem fixa também influenciam. O valor de eficiência acima de 1 para N=10000 com dois membros é uma observação desta campanha: efeitos de aquecimento, caches e variação podem produzir ganho aparente superlinear; não se conclui eficiência física superior a 100%. Não extrapolar para máquinas independentes nem confundir uma soma didática com benchmark de produção.

Correção, falha e expansão

N=0, N=1 e N=10 produziram 0, 1 e 55. N negativo e pedido com N diferente do carregado foram rejeitados. Para 100000 números, o total esperado é 5000050000.

O mesmo processo cliente permaneceu aberto durante queda e expansão. Uma rodada interrompida não é recuperada por partes: a próxima refaz a soma inteira. Recorte real do acompanhamento:

```
VALIDADO n=100000 soma=5000050000 membros=3 ms=2418.80086
RODADA_INTERROMPIDA ExecutionException; próxima rodada refaz a soma inteira
VALIDADO n=100000 soma=5000050000 membros=2 ms=3307.606102
RODADA_INTERROMPIDA IllegalStateException; próxima rodada refaz a soma inteira
VALIDADO n=100000 soma=5000050000 membros=3 ms=2779.31615
```

Eventos registrados:

- 2026-09-18T18:28:16.642033: FALHA_ABRUPTA, cliente PID 49305.

- 2026-09-18T18:28:29.375319: RECUPERACAO_VALIDADA, cliente PID 49305.

- 2026-09-18T18:28:29.393728: EXPANSAO_DURANTE_ACOMPANHAMENTO, cliente PID 49305.

- 2026-09-18T18:28:40.444630: EXPANSAO_VALIDADA, cliente PID 49305.

Partições antes e depois

```
17-estado.log
[127.0.0.1]:5701 primarias=91 seguro=true
[127.0.0.1]:5703 primarias=90 seguro=true
[127.0.0.1]:5702 primarias=90 seguro=true

19-estado.log
[127.0.0.1]:5701 primarias=136 seguro=true
[127.0.0.1]:5702 primarias=135 seguro=true

21-estado.log
[127.0.0.1]:5701 primarias=91 seguro=true
[127.0.0.1]:5704 primarias=90 seguro=true
[127.0.0.1]:5702 primarias=90 seguro=true
```

Reprodução e limites

Executar python3 scripts/experiments.py e python3 scripts/summarize.py. CSV bruto, logs e environment.json estão na pasta da campanha. Scripts de documentos e compilação LaTeX podem ser executados depois para atualizar os PDFs.

Testes demonstram falha de processo e recuperação no mesmo host. Não foram testados máquinas remotas, falha total do host, queda do cliente ou split-brain. O exemplo assume entrada imutável e conhecida.

Correspondência com o enunciado

Requisito	Atendimento na versão básica
Exemplo simples do site do framework	SomaTask, documentação oficial Hazelcast; adaptação identificada
Grupo de até três integrantes	Campo para nomes e matrículas no relatório e LaTeX; preencher
Item 1: usos, limites e mercado (~5 min)	Relatório §2 e primeiro bloco dos slides
Item 1: maior aplicação e quantidade de máquinas	Ausência de recorde comprovado explicitada; caso comercial 20 nós e benchmark 45 nós com fontes
Item 2: arquitetura e camadas (~5 min)	Relatório §3 e segundo bloco dos slides
Item 3: instalação de middleware	Não obrigatório na opção framework; guia de instalação local e alternativas adicionais
Item 4: origem, conceitos e aplicação	Soma distribuída / redução, SumTask oficial, código comentado e explicação introdutória
Item 4: execução em cluster	Processos Java independentes conectados por TCP; logs e somas parciais
Item 4: inputs maiores	1000, 10000 e 100000 números; 1, 2 e 3 membros; 27 medições
Item 4: queda abrupta	Processo membro encerrado sem shutdown; backup; nova rodada refaz toda a soma
Item 4: expansão sem reiniciar aplicação	Mesmo cliente acompanha saída e entrada de membro; pode haver pausa/rodada descartada
Item 5: responder questionários (~10 min)	Preparação com perguntas; questionário específico não incluído no enunciado
Apresentação fundamental	Fonte LaTeX/Beamer + PDF + notas e roteiro

Consulte RESULTADOS.md para saber exatamente o que foi executado e aprovado. Resultados usam JVMs no mesmo host. Máquinas remotas e falha completa de host não foram testadas. Os integrantes ainda precisam se identificar, ensaiar e realizar a apresentação no workshop.

Referências

1. **Hazelcast 5.6 — Java Executor Service, seção Implementing a Callable Task, exemplo SumTask.** Origem direta do exemplo adaptado. https://docs.hazelcast.com/hazelcast/5.6/computing/executor-service

2. **Hazelcast 5.6 — Making Your Map Data Safe.** Cópias síncronas, memória e recuperação dos dados. https://docs.hazelcast.com/hazelcast/5.6/data-structures/backing-up-maps

3. **Hazelcast — Data Partitioning (manual 5.0).** Explicação de partições e coordenação; distribuição também observada nos logs desta execução 5.6. https://docs.hazelcast.com/hazelcast/5.0/overview/data-partitioning

4. **Hazelcast — British Gas/Hive.** Caso publicado com 20 nós. https://hazelcast.com/resources/case-study-hazelcast-powers-internet-of-things-infrastructure/

5. **Hazelcast — BNP Paribas Bank Polska.** Caso de aplicação bancária. https://hazelcast.com/resources/case-study-achieving-ultra-low-latency-and-high-availability-in-online-banking/

6. **Hazelcast — Billion Events Per Second with Millisecond Latency.** Benchmark histórico Jet 4.3, 45 nós. Não comprova a maior aplicação mundial. https://hazelcast.com/blog/billion-events-per-second-with-millisecond-latency-streaming-analytics-at-giga-scale/